OIT

OIT(Order Independent Transparency)，主要用于解决大量复杂半透明物体渲染，按照正常深度排序的方案会出现几个问题：

- 一个较大的半透明物体附近有多个小的半透明物体，排序本身出现问题（可以把大物体拆成多个小物体）
- 物体存在自交叉
- 多个物体存在交叉

Related Work

1. Sorted Alpha Blending

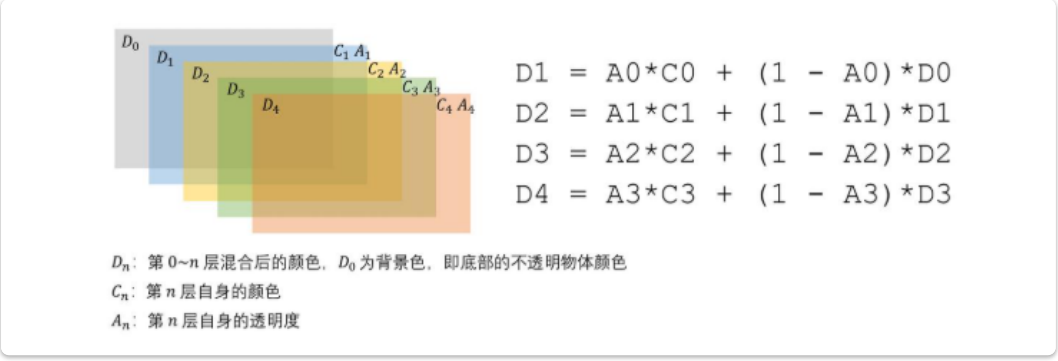

image-4.png

这里就是最常用的排序方案，着色公式直接使用

$$C_{final} = \sum_{i=1}^{N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

一般使用这个方案不会有什么问题，除非有一些上面提到的复杂情况

2. Depth Peeling

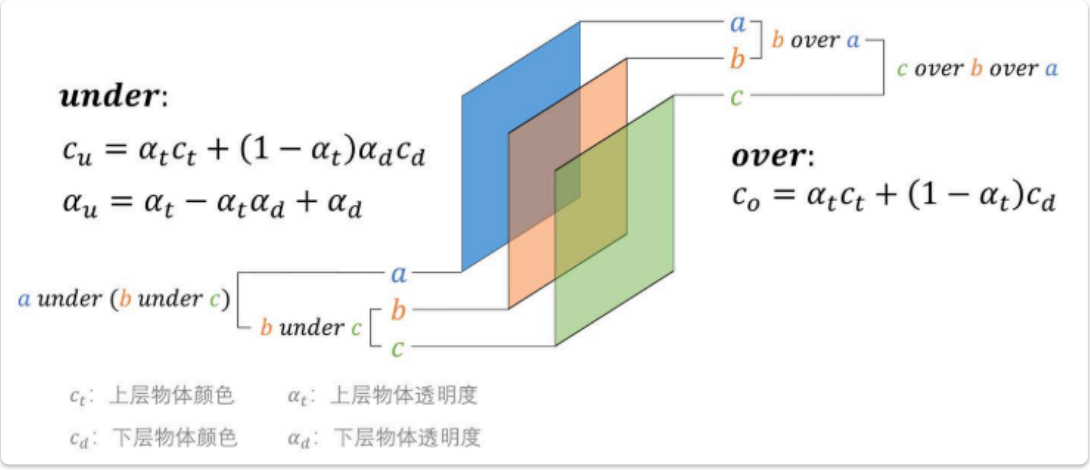

image-5.png

这个方案与 Sorted Alpha Blending 不同之处在于它是逆向的过程，每次绘制都会

除去上一次绘制的片元并找当前 Pass 离相机最近的片元进行绘制，相当于是排序方案的反向混合。

优势在于本质上解决了半透明的混合问题，从片元角度进行混合
劣势在于性能，渲染开销和需要剥离的层数有线性关系，适合用于离线渲染

3. Multi-Layer Alpha Blending

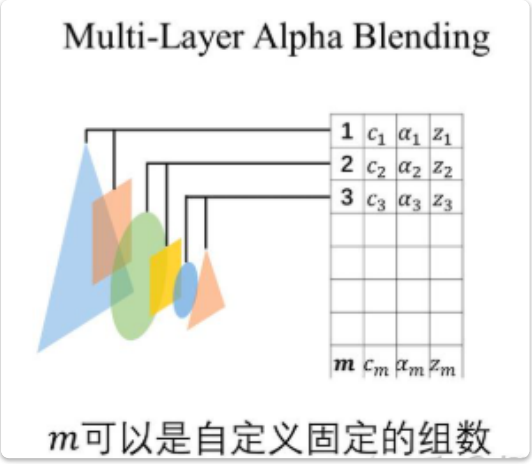

image-6.png

根据深度 z 将片元划分为多组，先对每组的片元混合后，只存储这一组的颜色、透明度和深度，最后对各组按顺序进行混合。

组内近似混合，组间精确混合

4. Weighted Sum & Weighted Average
这里的推导用了 Sorted Alpha Blending 的公式，把 D4 化简为 D0 的多项式，保留了低次项以及顺序无关项，Weighted Sum 的近似公式为

$$D_n = D_0(1 - \sum_{i=1}^{n} A_i) + \sum_{i=1}^{n} A_i C_i$$

Weighted Average的公式为

Weighted Sum	$D_n = D_0(1 - \sum_{i=1}^{n} A_i) + \sum_{i=1}^{n} A_i C_i$	D_0: 底部的不透明物体颜色 D_n: 第 0~n 层混合后的颜色 C_n: 第 n 层自身的颜色 A_n: 第 n 层自身的透明度
Weighted Average	$C_{sum} = \sum_{i=1}^{n} A_i C_i \quad A_{sum} = \sum_{i=1}^{n} A_i$	
	$C_{wavg} = \frac{C_{sum}}{A_{sum}} \quad A_{avg} = \frac{A_{sum}}{n} \quad u = (1 - A_{avg})^n$	
	$D_n = (1 - u)C_{wavg} + uD_0$	

image-8.png

效果如下

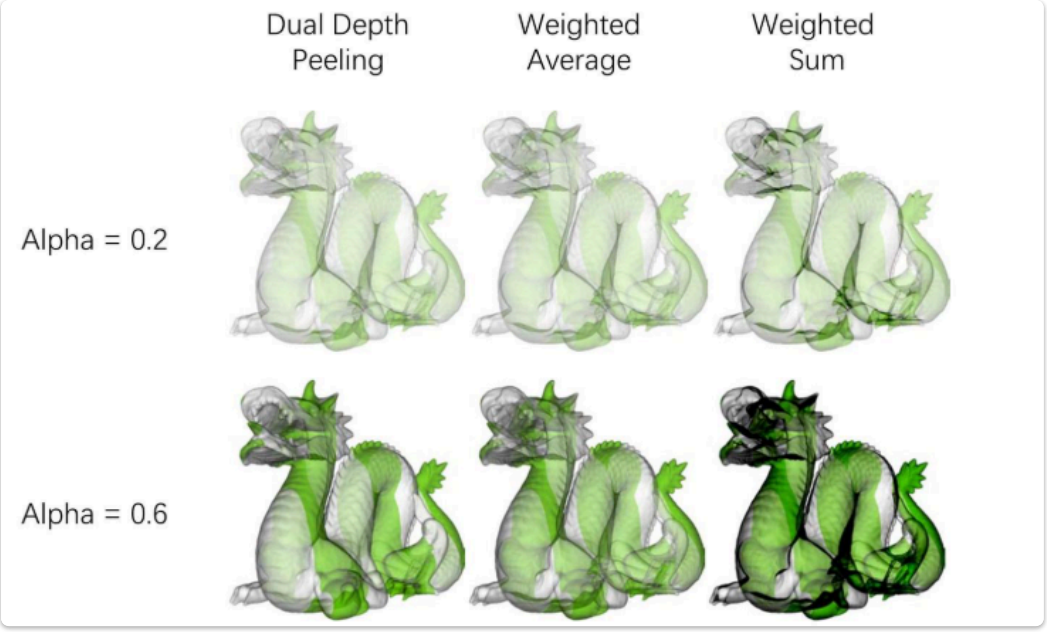

image-9.png

UE

Sorted Triangles

在Static Mesh的细节面板可以开启 Sorted Triangles，主要用于解决模型的自穿插

在 DeferredShadingRenderer 的 RenderTranslucency 前面调用 OIT::AddSortTrianglesPass来排序，做法是把三角面中心在Shader中转换为Mesh的深度然后排序，最后得到一个根据深度排序的 Index Buffer